ResTier

Tiered Storage Engine for ResilientDB

Master's Project Presentation
Under the guidance of Prof M. Sadoghi

Rajaram Manohar Joshi
MS in Computer Science

ExpoLab
Creativity Unfolded

Apache ResilientDB
Incubating

UCDAVIS
COMPUTER SCIENCE

What is ResilientDB

ResilientDB is a high-throughput distributed ledger built on scale-centric design principles, aimed at democratizing and decentralizing computation. Originally conceived at UC Davis's ExpoLab, it is now an Apache Incubating project.

Key features -

- Renewed the vision of Bitcoin/blockchain - integrating privacy, integrity, transparency, and accountability into the computational model
- Built around fault-tolerant BFT consensus protocols (to ensure the system continues operating correctly even when some nodes act maliciously or fail arbitrarily), redesigned for modern performance demands
- Uses parallelism and deep pipelining at every layer, optimized for modern hardware and global cloud infrastructure

Why ResTier?

Present Scenario

- All data stays in local LevelDB forever (SSD bloat)
- Storage grows unboundedly as the blockchain accumulates history (append-only by design)
- No mechanism to offload cold/archival data
- Historical data is less frequently queried

Why ResTier?

Proposed Solution

- Use IPFS as a cold tier to migrate historical data asynchronously based on configurable thresholds and checkpoints

Why IPFS?

- Content-addressed (CID = cryptographic hash of data)
- Trustless verification of content
- Deduplication built-in -> same content = same CID
- Peer-to-peer -> replicates across nodes automatically
- No vendor lock-in -> open source protocol

Problem Statement

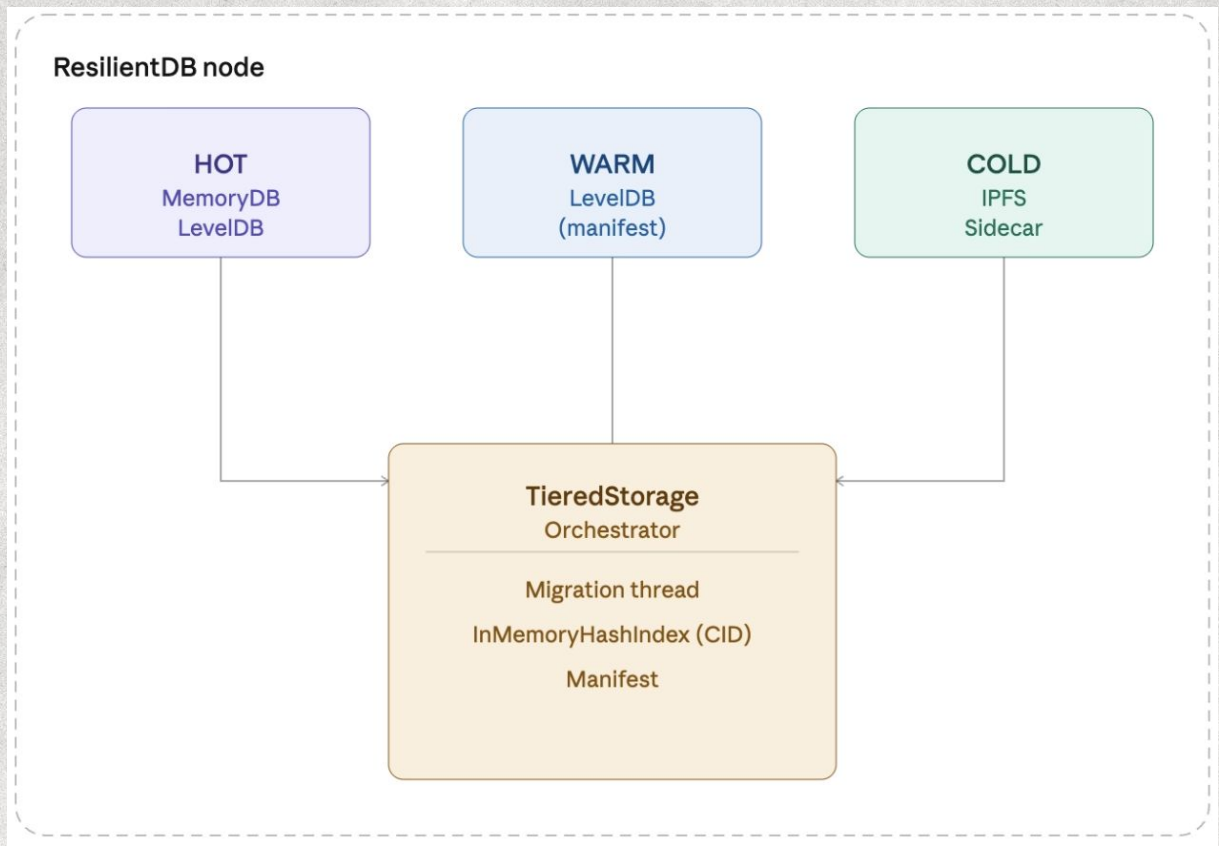
Design and implement a tiered storage engine for ResilientDB that:

1. Transparently migrates historical data from hot storage to IPFS
2. Maintains PBFT consensus latency (no write-path overhead)
3. Provides O(1) cold-read performance via an in-memory secondary index
4. Survives crashes with zero data loss
5. Operates in 4 storage modes to accommodate different deployment needs

Constraints:

- Each PBFT replica migrates independently (no cross-node consensus for migration)
- Zero data-loss window during migration
- Block cache must be invalidated on eviction (LevelDB hot tier)
- Minimal effect on write throughput and read latency

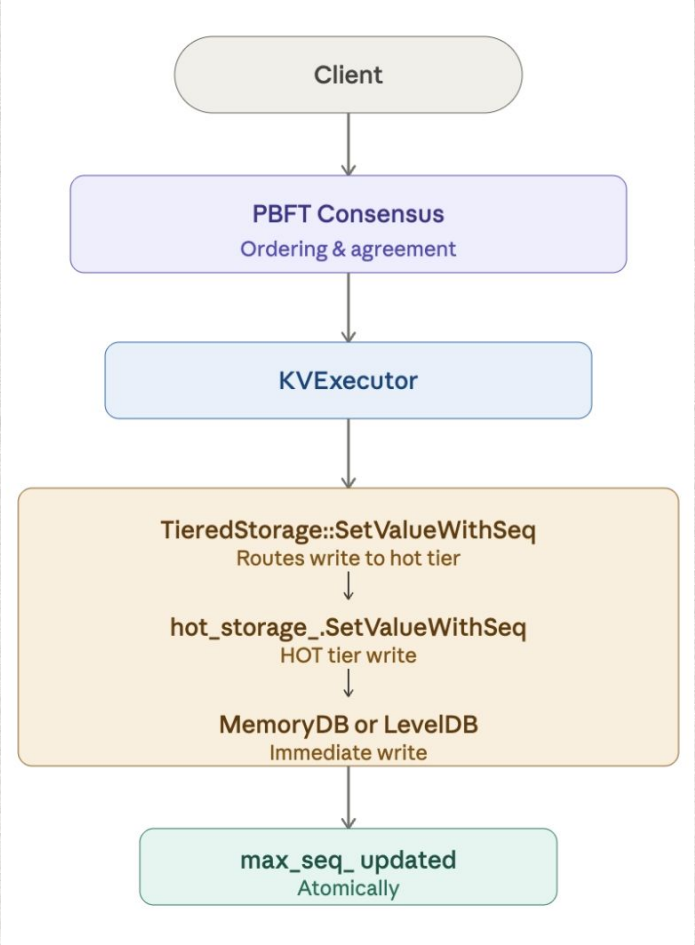
System Architecture



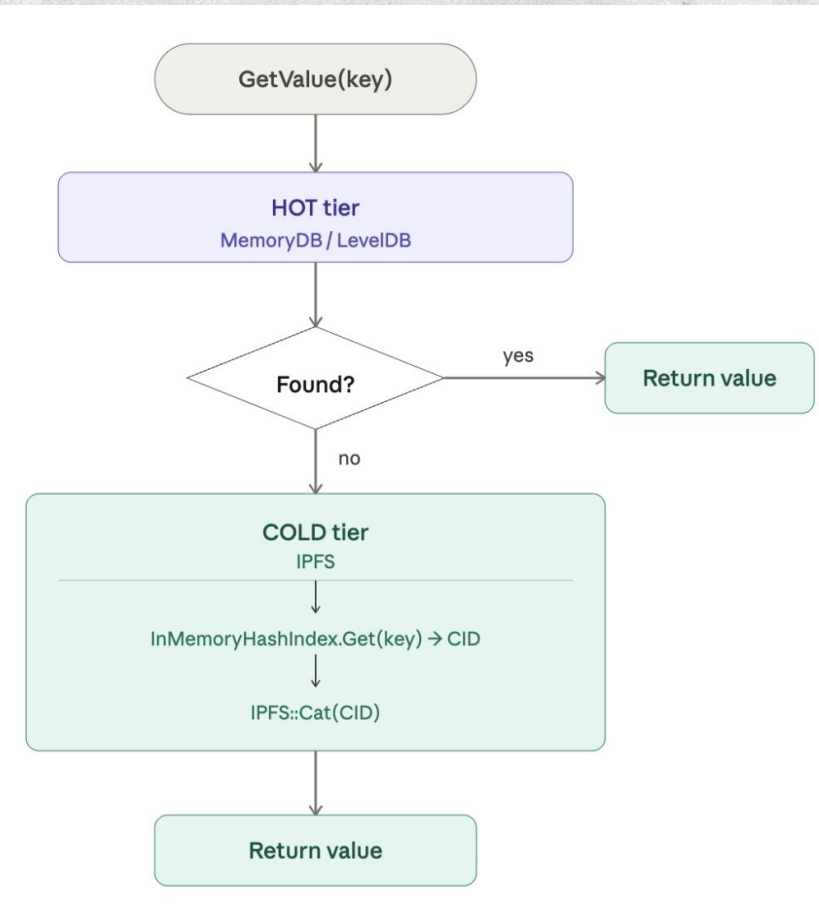
Four Storage Modes

Mode	Backend	Hot Tier	Warm Tier	Cold Tier	Use Case
0	MEMORYDB	MemoryDB	—	—	Dev/testing, fastest
1	LEVELDB	LevelDB	—	—	Production, small data
2	TIERED	LevelDB	LevelDB (manifest)	IPFS	Production, large data
3	TIERED	MemoryDB	LevelDB (manifest)	IPFS	High-throughput, crash-tolerant

Write Path (No Overhead)



Read Path (Fallback)



Crash Recovery

Four windows analyzed:

Window	State	GET behavior	Safe?
After IPFS upload, before index add	Data in IPFS + hot, NOT in index	Hits hot → correct	✅
After index add, before hot delete	Data in IPFS + index + hot	Hits hot → correct	✅
After hot delete	Data in IPFS + index only	Index→CID→IPFS Cat	✅
After hot delete, LRU cache stale	Stale entry in block cache	**Was: stale value. Fixed:** `Remove(key)`	✅

Zero data-loss. At worst, a key exists in both hot and cold (safe duplicate).

Conclusion

ResTier delivers:

- Transparent hot→cold data migration with zero write-path overhead
- O(1) cold-read performance via in-memory secondary index
- Safe concurrent reads during migration — 0 mismatches verified
- Crash recovery with no data loss
  4 storage modes for different deployment needs
- Works with existing PBFT consensus no consensus changes

Thank You

Rajaram Manohar Joshi
MS in Computer Science